Calibration Synthesizer

Assignment Explanation

Name: Sneha Gautam
File: calibration_synthesizer.py
Date: 31.08.2026

---

1. Introduction

The program processes a calibration feed containing multiple batches of sensor readings. Each batch contains a calibration ID followed by numerical readings or control signals.

The program performs the following tasks:

- Processes batches one by one.
- Detects and skips empty batches.
- Traverses readings from right to left.
- Ignores readings marked as `"IGNORE"`.
- Suppresses a batch when `"FAULT"` is encountered.
- Completely terminates processing when `"HALT"` is encountered.
- Counts all valid floating-point readings.
- Finds the global maximum reading manually.
- Finds the global minimum reading manually.
- Calculates a calibration checksum for batches that complete normally.

2. Source Code

The complete Python program is shown below.

```
calibration_feed = [
    [201, 6.0, 9.5, "IGNORE", 4.0],
    [],
    [202, 11.2, "FAULT", 7.8, 5.5],
    [203, 14.0, 3.5, 8.25],
    [204, 2.75, "HALT", 6.0]
]

feed_cursor = 0
total = 0
```

```
maximum = None
minimum = None
checksum = 0
shutdown = False


while (current_slice := calibration_feed[feed_cursor:feed_cursor
   + 1]):

    batch = current_slice[0]

    # Check empty batch
    if not batch:
        print("BATCH", feed_cursor, "is EMPTY. Proceeding.")
        feed_cursor += 1
        continue

    # Calibration ID
    batch_id = batch[0]

    print("Evaluating Batch", feed_cursor,
          "(ID :", batch_id, ")...")


    # Find length manually
    batch_length = 0

    for i in batch:
        batch_length += 1


    # SUM
    batch_sum = 0.0


    # Read from right to left
    for i in range(1, batch_length):

        index = batch_length - i
        reading = batch[index]


        # IGNORE
        if reading == "IGNORE":
            print("Signal IGNORE encountered at batch", batch_id)
            continue


        # FAULT
        if reading == "FAULT":
            print("Signal FAULT detected. Suppressing batch",
```

```
                batch_id)
            break


        # HALT
        if reading == "HALT":
            print("Signal HALT detected. Executing emergency
                protocol.")
            shutdown = True
            break


        # Number Reading
        if isinstance(reading, float):

            batch_sum = batch_sum + reading
            total = total + 1


            # Find maximum
            if maximum is None:
                maximum = reading

            elif reading > maximum:
                maximum = reading


            # Find minimum
            if minimum is None:
                minimum = reading

            elif reading < minimum:
                minimum = reading


    # This else belongs to the FOR loop
    else:

        if batch_id % 2 == 0:
            batch_sum = batch_sum * 1.5

        else:
            batch_sum = batch_sum * 0.8

        checksum = checksum + batch_sum


    # Stop everything if HALT found
    if shutdown:
        break

```

```
    feed_cursor += 1


# Final output
print()
print("========================================")

if shutdown:
    print("CALIBRATION COMPLETE : EMERGENCY TERMINATION")

else:
    print("CALIBRATION COMPLETE : NORMAL COMPLETION")

print("========================================")

print("Total Valid Readings Processed :", total)
print("Global Calibration Checksum :", checksum)
print("Maximum Reading Encountered :", maximum)
print("Minimum Reading Encountered :", minimum)
```

3. Detailed Line-by-Line Explanation

3.1 Calibration Feed

```
calibration_feed = [
    [201, 6.0, 9.5, "IGNORE", 4.0],
    [],
    [202, 11.2, "FAULT", 7.8, 5.5],
    [203, 14.0, 3.5, 8.25],
    [204, 2.75, "HALT", 6.0]
]
```

This creates a list containing five calibration batches.

The first element of every non-empty batch is its calibration ID.

For example:

```
[201, 6.0, 9.5, "IGNORE", 4.0]
```

Here, `201` is the calibration ID.

The remaining elements are readings or control signals.

The second batch is:

```
[]
```

which is an empty batch.

3.2 Variable Initialization

```
feed_cursor = 0
```

Stores the index of the current batch.

```
total = 0
```

Stores the total number of valid floating-point readings processed.

```
maximum = None
minimum = None
```

These variables store the global maximum and minimum values.

They initially contain `None` because no reading has been processed yet.

```
checksum = 0
```

Stores the total calibration checksum.

```
shutdown = False
```

This Boolean variable indicates whether an emergency `"HALT"` signal has been detected.

Initially, there is no emergency, so it is set to `False`.

4. Outer While Loop

```
while (current_slice := calibration_feed[feed_cursor:feed_cursor
    + 1]):
```

This is the main loop of the program.

It uses the assignment expression operator, also called the **walrus operator**:

```
:=
```

The current batch is obtained using slicing.

For example, when:

```
feed_cursor = 0
```

the expression:

```
calibration_feed[0:1]
```

produces a list containing the first batch.

The result is stored in `current_slice`.

When the cursor moves beyond the available batches, the slice becomes an empty list. An empty list is considered false in Python, causing the `while` loop to terminate.

5. Getting the Batch

```
batch = current_slice[0]
```

The slice contains one batch, so index 0 retrieves that batch.

For example:

```
current_slice =
[[201, 6.0, 9.5, "IGNORE", 4.0]]
```

After:

```
batch = current_slice[0]
```

we get:

```
[201, 6.0, 9.5, "IGNORE", 4.0]
```

6. Empty Batch Check

```
if not batch:
```

Checks whether the current batch is empty.

For Batch 1:

```
batch = []
```

Therefore, the condition is true.

```
print("BATCH", feed_cursor, "is EMPTY. Proceeding.")
```

prints a message informing the user that the batch is empty.

```
feed_cursor += 1
```

moves to the next batch.

```
continue
```

skips the rest of the current iteration and starts the next iteration of the `while` loop.

7. Calibration ID

```
batch_id = batch[0]
```

The first element of the batch is stored as the calibration ID.

For example:

```
batch = [201, 6.0, 9.5, "IGNORE", 4.0]
```

gives:

```
batch_id = 201
```

The program then prints the batch being evaluated.

8. Manual Length Calculation

```
batch_length = 0
```

Initializes a counter.

The program does not use Python's `len()` function.

Instead, it calculates the length manually.

```
for i in batch:
    batch_length += 1
```

The loop visits every element in the batch and increments `batch_length`.

For:

```
[201, 6.0, 9.5, "IGNORE", 4.0]
```

the final value is:

$$\texttt{batch\_length} = 5$$

9. Batch Sum

```
batch_sum = 0.0
```

This stores the sum of valid readings for the current batch.

It is reset to zero whenever a new batch starts.

10. Backward Traversal

```
for i in range(1, batch_length):
```

This loop processes the readings from right to left.

The first element of a batch is the calibration ID, so it is not processed.

The actual index is calculated using:

```
index = batch_length - i
```

For a batch of length 5:

i	index
1	4
2	3
3	2
4	1

Therefore, the readings are processed in reverse order.

11. Reading Selection

```
reading = batch[index]
```

Retrieves the reading at the calculated index.

For Batch 0, the order becomes:

```
4.0
"IGNORE"
9.5
6.0
```

This is right-to-left traversal.

12. IGNORE Signal

```
if reading == "IGNORE":
    print("Signal IGNORE encountered at batch", batch_id)
    continue
```

When the reading is `"IGNORE"`, the program prints a message.

The `continue` statement skips that reading.

Therefore, `"IGNORE"` does not affect:

- The batch sum
- The total reading count
- The maximum
- The minimum

The loop then continues with the next reading.

13. FAULT Signal

```
if reading == "FAULT":
    print("Signal FAULT detected. Suppressing batch", batch_id)
    break
```

When `"FAULT"` is found, the program prints a message.

The `break` statement terminates the current inner `for` loop.

The outer `while` loop continues processing subsequent batches.

Therefore, a FAULT suppresses only the current batch's checksum.

14. HALT Signal

```
if reading == "HALT":
    print("Signal HALT detected. Executing emergency protocol.")
    shutdown = True
    break
```

When `"HALT"` is encountered, the program sets:

```
shutdown = True
```

It then breaks out of the inner `for` loop.

The `shutdown` variable is later checked outside the inner loop to stop the outer `while` loop as well.

Therefore, HALT causes complete termination of feed processing.

15. Valid Number Reading

```
if isinstance(reading, float):
```

This checks whether the current reading is a floating-point number.

Only floating-point readings are processed as valid readings.

```
batch_sum = batch_sum + reading
```

Adds the valid reading to the current batch sum.

```
total = total + 1
```

Increases the global count of valid readings.

16. Finding the Maximum

```
if maximum is None:
    maximum = reading
```

If no valid reading has been encountered yet, the current reading becomes the maximum.

For subsequent readings:

```
elif reading > maximum:
    maximum = reading
```

If the current reading is larger than the stored maximum, the maximum is updated.

This manually performs the job of Python's `max()` function.

17. Finding the Minimum

```
if minimum is None:
    minimum = reading
```

The first valid reading becomes the initial minimum.

For subsequent readings:

```
elif reading < minimum:
    minimum = reading
```

If a smaller reading is encountered, the minimum is updated.

This manually performs the job of Python's `min()` function.

18. For-Else Structure

The following `else` belongs to the inner `for` loop:

```
for i in range(1, batch_length):
    ...
else:
    ...
```

A Python `for-else` executes the `else` block only if the loop finishes normally without encountering `break`.

Therefore:

Signal	For-else Executes?
IGNORE	Yes
FAULT	No
HALT	No
Normal completion	Yes

19. Checksum Calculation

```
if batch_id % 2 == 0:
    batch_sum = batch_sum * 1.5
```

The modulo operator `%` calculates the remainder.

If the calibration ID is even, the batch sum is multiplied by 1.5.

For example:

$$202 \bmod 2 = 0$$

so ID 202 is even.

For odd IDs:

```
else:
    batch_sum = batch_sum * 0.8
```

the batch sum is multiplied by 0.8.

Finally:

```
checksum = checksum + batch_sum
```

adds the adjusted batch sum to the global checksum.

20. Emergency Stop

```
if shutdown:
    break
```

This statement is outside the inner `for` loop.

If a HALT signal was found, `shutdown` is `True`.

Therefore, this `break` terminates the outer `while` loop.

This is how the program completely stops after a HALT signal.

21. Moving to the Next Batch

```
feed_cursor += 1
```

If no emergency shutdown occurred, the cursor is increased by one.

For example:

$$0 \to 1 \to 2 \to 3 \to 4$$

This allows the next batch to be processed.

22. Execution Trace

22.1 Batch 0

Batch 0 is:

```
[201, 6.0, 9.5, "IGNORE", 4.0]
```

The readings are processed backwards:

```
4.0 -> IGNORE -> 9.5 -> 6.0
```

Valid readings are:

$$4.0 + 9.5 + 6.0 = 19.5$$

ID 201 is odd:

$$19.5 \times 0.8 = 15.6$$

Therefore:

$$\texttt{checksum} = 15.6$$

The number of valid readings is 3.

22.2 Batch 1

Batch 1 is empty:

```
[]
```

It is skipped using `continue`.

No reading is processed.

22.3 Batch 2

Batch 2 is:

```
[202, 11.2, "FAULT", 7.8, 5.5]
```

The readings are processed backwards:

```
5.5 -> 7.8 -> FAULT
```

The two valid readings are processed.

When `FAULT` is found, the inner loop breaks.

Therefore, the loop's `else` does not execute.

The partial batch sum is not added to the checksum.

22.4 Batch 3

Batch 3 is:

```
[203, 14.0, 3.5, 8.25]
```

The readings are processed backwards:

```
8.25 -> 3.5 -> 14.0
```

All three readings are valid.

Their sum is:

$$8.25 + 3.5 + 14.0 = 25.75$$

ID 203 is odd:

$$25.75 \times 0.8 = 20.6$$

The checksum becomes:

$$15.6 + 20.6 = 36.2$$

22.5 Batch 4

Batch 4 is:

```
[204, 2.75, "HALT", 6.0]
```

The readings are processed backwards:

```
6.0 -> HALT
```

The value 6.0 is processed.

When `HALT` is encountered:

```
shutdown = True
```

The inner loop breaks.

Then the outer loop also breaks because:

```
if shutdown:
    break
```

Therefore, the program terminates.

The value 2.75 is never processed.

23. Final Values

The total valid readings are:

$$3 + 0 + 2 + 3 + 1 = 9$$

Therefore:

$$\boxed{\text{Total Valid Readings} = 9}$$

The checksum is:

$$15.6 + 20.6 = 36.2$$

Therefore:

$$\boxed{\text{Checksum} = 36.2}$$

The maximum valid reading encountered is:

$$\boxed{14.0}$$

The minimum valid reading encountered is:

$$\boxed{3.5}$$

24. Expected Output

```
BATCH 1 is EMPTY. Proceeding.
Evaluating Batch 0 (ID : 201 )...
Signal IGNORE encountered at batch 201
Evaluating Batch 2 (ID : 202 )...
Signal FAULT detected. Suppressing batch 202
Evaluating Batch 3 (ID : 203 )...
Evaluating Batch 4 (ID : 204 )...
Signal HALT detected. Executing emergency protocol.

========================================
CALIBRATION COMPLETE : EMERGENCY TERMINATION
========================================
Total Valid Readings Processed : 9
Global Calibration Checksum : 36.2
Maximum Reading Encountered : 14.0
Minimum Reading Encountered : 3.5
```

Note: The exact placement of the Batch 1 message depends on the program execution order. The important final values are:

Total Valid Readings	9
Global Calibration Checksum	36.2
Maximum Reading	14.0
Minimum Reading	3.5

25. Important Python Concepts Used

1. **Walrus Operator:** `:=` assigns and evaluates a value in the same expression.
2. **List Slicing:** `calibration_feed[start:end]` extracts a portion of a list.
3. **Manual Length Calculation:** The program counts elements using a loop instead of `len()`.
4. **Backward Traversal:** The index is calculated using `batch_length - i`.
5. **continue:** Skips the current iteration.
6. **break:** Terminates the current loop.
7. **For-Else:** The `else` executes only when the `for` loop finishes without a `break`.
8. **Modulo Operator:** `%` is used to determine whether the calibration ID is even or odd.
9. **Boolean Flag:** `shutdown` communicates an emergency condition from the inner loop to the outer loop.
10. **isinstance():** Checks whether a reading is a floating-point number.

26. Conclusion

The program demonstrates controlled processing of calibration batches using nested loops and conditional control flow.

The most important logic is the distinction between the three control signals:

Signal	Action
IGNORE	Skip the current reading
FAULT	Stop processing the current batch
HALT	Stop the entire calibration feed

The program finally produces:

```
Total Valid Readings = 9
Global Calibration Checksum = 36.2
Maximum Reading = 14.0
Minimum Reading = 3.5
```